

Template Method & Strategy Patterns in Java Spring Boot Microservices

Template Method & Strategy Patterns in Java Spring Boot Microservices (0-7 Years Guide)

PART 1 — TEMPLATE METHOD PATTERN

1. What It Is

Template Method defines the **skeleton of an algorithm** in a base class/method, with fixed steps in a fixed order, but lets certain steps be customized by the caller (via a callback, subclass override, or lambda) without changing the overall structure.

Why it matters: eliminates repetitive boilerplate (resource open/close, exception translation, connection management) so you only write the part that's actually specific to your use case.

2. Classic Java Implementation (Without Spring)

```
public abstract class ReportGenerator {

    // template method – defines the fixed algorithm skeleton
    public final void generate() {
        openConnection();
        fetchData();
        formatReport(); // varies per subclass
        closeConnection();
    }

    private void openConnection() { System.out.println("Opening
connection"); }
    private void closeConnection() { System.out.println("Closing
connection"); }
    private void fetchData() { System.out.println("Fetching data");
}

    protected abstract void formatReport(); // subclass fills in the
variable step
    }

    public class PdfReportGenerator extends ReportGenerator {
        protected void formatReport() { System.out.println("Formatting
as PDF"); }
    }
```

The caller never has to remember to open/close the connection — only implement the one part that differs.

3. How JdbcTemplate Uses This Pattern

Without JdbcTemplate, every query requires manually opening a connection, creating a statement, executing it, mapping the ResultSet, handling SQLException, and closing everything in a finally block. JdbcTemplate handles **all of that fixed boilerplate internally**, and you only supply the variable part — the SQL and the row-mapping logic.

```
@Repository
public class OrderJdbcRepository {
    private final JdbcTemplate jdbcTemplate;

    public OrderJdbcRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public List<Order> findByStatus(String status) {
        String sql = "SELECT * FROM orders WHERE status = ?";
        return jdbcTemplate.query(sql, (rs, rowNum) ->
            new Order(rs.getLong("id"), rs.getString("customer_id"),
rs.getDouble("amount")),
            status
        );
    }
}
```

Internally, JdbcTemplate.query(...) follows this fixed skeleton: 1. Get a connection from the pool 2. Create/prepare the statement, bind parameters 3. Execute the query 4. Iterate the ResultSet, calling **your** RowMapper lambda for each row (the variable step) 5. Translate any SQLException into Spring's unchecked DataAccessException hierarchy 6. Release the connection back to the pool — even on failure

You only ever write step 4's logic (the RowMapper); everything else is the "template."

4. How RestTemplate Uses This Pattern

RestTemplate's internals similarly follow a fixed request lifecycle, letting you plug in only the variable parts:

```
RestTemplate restTemplate = new RestTemplate();

Order order = restTemplate.execute(
    "https://order-service/orders/{id}",
    HttpMethod.GET,
    request -> request.getHeaders().add("Authorization", "Bearer " +
token), // request callback (variable)
    (response) -> objectMapper.readValue(response.getBody(),
Order.class), // response extractor (variable)
    orderId
);
```

Fixed skeleton internally: 1. Resolve the URI template and variables 2. Open the HTTP connection via the configured ClientHttpRequestFactory 3. Apply your RequestCallback to customize headers/body (variable step) 4. Execute the request 5. Apply your ResponseExtractor/HttpMessageConverter to parse the response (variable step) 6. Handle errors via ResponseErrorHandler 7. Close the connection

In everyday code you rarely touch this low-level API — you use the higher-level convenience methods (`getForObject`, `postForEntity`, `exchange`), which internally delegate to this same template skeleton.

```
// What you actually write day to day
Order order = restTemplate.getForObject("https://order-
service/orders/{id}", Order.class, orderId);
```

5. Other Spring “Template” Classes (Same Pattern, Different Resource)

Template class	Fixed steps it manages	Variable part you supply
JdbcTemplate	Connection/statement lifecycle, exception translation	SQL + RowMapper
RestTemplate / WebClient	HTTP connection lifecycle, serialization	URL, headers, body, response type
KafkaTemplate	Producer lifecycle, partition/broker handling	Topic + message payload
RedisTemplate	Connection lifecycle, serialization	Key/value operations
TransactionTemplate	Transaction begin/commit/rollback	The business logic inside <code>execute()</code>

```
@Service
public class OrderService {
    private final TransactionTemplate transactionTemplate;

    public void placeOrder(Order order) {
        transactionTemplate.execute(status -> {
            // your variable business logic – commit/rollback
            // handled by the template
            orderRepository.save(order);
            inventoryService.reserve(order);
            return null;
        });
    }
}
```

6. Where Template Method Shows Up in Microservices

- Any place with a repetitive “setup → do variable work → teardown/cleanup” shape
- Retry wrappers (fixed retry loop, variable operation)
- Audit logging wrappers (fixed log-before/log-after, variable business call)
- Batch processing skeletons (fixed chunking/commit logic, variable item processing — Spring Batch’s `ItemProcessor` is built this way)

```
public abstract class BaseSyncJob {
    public final void run() {
        log.info("Sync started");
        try {
            doSync(); // variable step
        } catch (Exception e) {
            log.error("Sync failed", e);
            throw e;
        } finally {
            log.info("Sync finished");
        }
    }
}
```

```

    }

    protected abstract void doSync();
}

```

7. Template Method — By Experience Level

0-2 Years

Expected: Use JdbcTemplate/RestTemplate for basic queries/calls, know they reduce boilerplate. **Common questions:** What does JdbcTemplate do that plain JDBC doesn't? Write a basic query using JdbcTemplate with a RowMapper.

2-4 Years

Expected: Understand RowMapper vs ResultSetExtractor, use RestTemplate's exchange() for custom headers/methods, understand why RestTemplate is now in maintenance mode in favor of WebClient. **Common questions:** Difference between RowMapper and ResultSetExtractor? Why is WebClient preferred over RestTemplate in new projects? How does Spring translate a SQLException into a DataAccessException?

4-7 Years

Expected: Recognize Template Method as the underlying design of JdbcTemplate/RestTemplate/TransactionTemplate/KafkaTemplate; design your own template-style utility class for a repeated cross-cutting concern (e.g., a RetryTemplate-style wrapper, or a metrics-wrapped execution template); understand the trade-off of blocking RestTemplate vs reactive, non-blocking WebClient in a high-throughput microservice; know when NOT to use a template abstraction (over-engineering a one-off operation). **Common questions:** How is JdbcTemplate an example of the Template Method pattern — walk through its internal execution flow? Why would you build a custom "template" utility class for your team, and what would go in the fixed vs variable parts? Compare blocking (RestTemplate) vs non-blocking (WebClient) call patterns under high concurrent load in a microservice — how does the internal template-style design differ?

PART 2 — STRATEGY PATTERN

1. What It Is

Strategy defines a **family of interchangeable algorithms**, encapsulates each one behind a common interface, and lets the client choose/swap which one to use at runtime — without if/else or switch chains scattered through business logic.

Why it matters: new algorithms/behaviors can be added without touching existing code (Open/Closed Principle); each algorithm is independently testable.

2. Classic Java Implementation

```

public interface DiscountStrategy {
    double apply(double price);
}

```

```

    }

    public class FestiveDiscount implements DiscountStrategy {
        public double apply(double price) { return price * 0.9; } // 10%
off
    }

    public class LoyaltyDiscount implements DiscountStrategy {
        public double apply(double price) { return price * 0.85; } //
15% off
    }

    public class NoDiscount implements DiscountStrategy {
        public double apply(double price) { return price; }
    }

    public class PricingEngine {
        private final DiscountStrategy strategy;

        public PricingEngine(DiscountStrategy strategy) { this.strategy
= strategy; }

        public double calculatePrice(double basePrice) {
            return strategy.apply(basePrice);
        }
    }
}

```

Usage:

```

PricingEngine engine = new PricingEngine(new FestiveDiscount());
double finalPrice = engine.calculatePrice(1000);

```

3. Spring Boot Idiomatic Version — Strategy + Factory Combined

This is the pattern you'll actually use in production — Spring auto-discovers all strategy beans, so adding a new one requires no changes anywhere else.

```

public interface PaymentStrategy {
    String type();
    void pay(double amount);
}

@Component("UPI")
public class UpiPayment implements PaymentStrategy {
    public String type() { return "UPI"; }
    public void pay(double amount) { System.out.println("Paid via
UPI: " + amount); }
}

@Component("CARD")
public class CardPayment implements PaymentStrategy {
    public String type() { return "CARD"; }
    public void pay(double amount) { System.out.println("Paid via
Card: " + amount); }
}

@Service
public class PaymentService {
    private final Map<String, PaymentStrategy> strategies;

    public PaymentService(List<PaymentStrategy> strategyList) {
        this.strategies = strategyList.stream()
            .collect(Collectors.toMap(PaymentStrategy::type, s ->
s));
    }
}

```

```

        public void processPayment(String type, double amount) {
            PaymentStrategy strategy = strategies.get(type);
            if (strategy == null) throw new
IllegalArgumentException("Unsupported payment type: " + type);
            strategy.pay(amount);
        }
    }
}

```

Adding **NetBanking** later requires zero changes to `PaymentService` — just a new `@Component("NETBANKING")` class.

4. Using Java's Functional Interfaces as Lightweight Strategies

For simple cases you don't always need a full interface hierarchy — a `Function`/`BiFunction` works as a strategy:

```

Map<String, Function<Double, Double>> discountStrategies = Map.of(
    "FESTIVE", price -> price * 0.9,
    "LOYALTY", price -> price * 0.85,
    "NONE", price -> price
);

double finalPrice = discountStrategies.get("FESTIVE").apply(1000.0);

```

5. Where Strategy Is Used in Real Microservices

- **Payment gateway selection** (UPI/Card/Wallet/NetBanking)
- **Discount/pricing rules** based on customer tier, coupon type, or campaign
- **Shipping cost calculation** strategies (standard/express/international)
- **Notification channel selection** (Email/SMS/Push — same beans, different interface role than Factory example)
- **Validation rule sets** that vary by tenant/region/regulatory requirement
- **Tax calculation** strategies varying by country/state

```

public interface TaxStrategy {
    String country();
    double calculateTax(double amount);
}

@Component("IN")
public class IndiaGstStrategy implements TaxStrategy {
    public String country() { return "IN"; }
    public double calculateTax(double amount) { return amount *
0.18; }
}

@Component("US")
public class UsSalesTaxStrategy implements TaxStrategy {
    public String country() { return "US"; }
    public double calculateTax(double amount) { return amount *
0.07; }
}

```

6. Strategy vs Factory — Common Point of Confusion (Interview Favorite)

Strategy	Factory
----------	---------

Solves	“Which algorithm/behavior to execute”	“Which object to create”
Focus	Behavior selection	Object instantiation
Typical combo	Often combined : a Factory looks up and returns the right Strategy bean	Factory can create Strategy instances, Singletons, or any other object type

In the Spring examples above, PaymentService’s map-lookup is technically doing **both**: it’s selecting a Strategy implementation using Factory-style bean lookup. This dual nature is a very common senior-level clarifying question.

7. Strategy — By Experience Level

0-2 Years

Expected: Define the pattern, implement a basic interface + 2 implementations + a client class. **Common questions:** What is the Strategy pattern? Give a real-world example. How is it different from just using if/else?

2-4 Years

Expected: Implement the Spring-idiomatic map-based strategy resolver; explain Open/Closed Principle benefit; refactor an existing if/else/switch chain into Strategy. **Common questions:** Design a system to support multiple payment methods cleanly. How would you add a new discount type without modifying existing code? What’s the difference between Strategy and State pattern (commonly confused)?

4-7 Years

Expected: Combine Strategy with Factory for dynamic resolution at scale (50+ strategies without one bloated class); apply Strategy across tenant/region-specific business rules in a multi-tenant microservice; understand performance implications of building the strategy map at startup (eager) vs on-demand (lazy) with many beans; design fallback/default strategy handling gracefully in production; test strategy implementations in isolation with clear contract-based unit tests. **Common questions:** How would you design a multi-tenant system where 50 different customers each need slightly different pricing/discount rules, without an unmanageable if/else chain? How do you handle an unregistered/unknown strategy type gracefully in production (fail-fast vs fallback default)? Compare Strategy pattern to Spring Profiles/@ConditionalOnProperty for swapping behavior — when would you use each? How does Strategy support the Open/Closed and Dependency Inversion principles simultaneously?

Quick Summary Table

Pattern	Core Purpose	Key Spring Example
Template Method	Fixed algorithm skeleton, variable steps plugged in	JdbcTemplate, RestTemplate, TransactionTemplate, KafkaTemplate
	Interchangeable	Payment/discount/tax strategy beans

How They Often Appear Together (Senior-Level Integration Point)

“Your PaymentService needs to call an external gateway per payment type, log every attempt the same way, and handle retries the same way regardless of which gateway is used — how would you design this?”

- **Strategy** picks which gateway implementation runs (UpiPayment, CardPayment, etc.)
- **Template Method** (e.g., a custom execute() wrapper, or Spring’s RetryTemplate) wraps every strategy call with the same fixed logging/retry/error-handling skeleton, so no strategy implementation has to repeat that boilerplate itself.

```
@Service
public class PaymentExecutor {
    private final Map<String, PaymentStrategy> strategies;
    private final RetryTemplate retryTemplate;

    public void execute(String type, double amount) {
        retryTemplate.execute(context -> {           // Template
            // Method – fixed retry skeleton
            strategies.get(type).pay(amount);        // Strategy –
            // variable algorithm
        });
    }
}
```